

[NOM DE L'UNIVERSITE / INSTITUT SUPERIEUR]

[Faculte / Ecole - Departement d'Informatique et Génie Logiciel]

MÉMOIRE DE FIN D'ÉTUDES

En vue de l'obtention de la Licence en Informatique / Génie Logiciel

CLEANCITY

Une plateforme mobile et web pour la gestion participative

des déchets urbains au Cameroun

« Collect . Recover . Preserve »

Présente par :

[Prenom NOM de l'étudiant(e)]

Sous l'encadrement de :

[Titre et Nom de l'encadreur académique]

Année académique [2025 - 2026]

Soutenu le [date de soutenance]

Dedicace

[Section a personnaliser par l'étudiant.]

A ma famille, pour son soutien constant tout au long de ce parcours académique,

A mes enseignants, pour la rigueur et les connaissances transmises,

A mes camarades de promotion, pour leur solidarité,

Je dedie ce travail.

Remerciements

Au terme de la rédaction de ce mémoire, je tiens à exprimer ma sincère gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Mes remerciements s'adressent en premier lieu à [Nom de l'encadreur académique], pour son encadrement rigoureux, sa disponibilité et ses conseils avisés qui ont guidé chaque étape de ce travail, de la conception à la rédaction finale.

Je remercie également l'ensemble du corps professoral et administratif de [Nom de l'université / institut], dont l'enseignement m'a permis d'acquérir les fondements théoriques et pratiques nécessaires à la conduite de ce projet.

Mes remerciements vont aussi aux membres du jury, qui me font l'honneur d'évaluer ce travail, ainsi qu'à toute personne ayant accepté de participer aux échanges et tests informels autour de l'application CleanCity.

Enfin, une pensée particulière pour ma famille et mes proches, pour leur soutien moral et leur patience durant toute la durée de ce projet.

Résumé

Les villes camerounaises font face à une pression croissante liée à la gestion des déchets solides urbains, conséquence directe d'une urbanisation rapide et d'une insuffisance des dispositifs numériques de suivi de la salubrité publique. Le signalement d'un point de décharge sauvage, la coordination des équipes de collecte et le suivi du traitement des déchets reposent encore largement sur des circuits informels, peu traçables et difficilement mesurables par les autorités municipales.

Le présent mémoire documenté la conception, le développement et le déploiement de CleanCity, une application mobile et web multiplateforme permettant de mettre en relation trois catégories d'acteurs -- les citoyens générateurs de déchets, les collecteurs et les centres de tri -- autour d'un cycle complet de signalement, de géolocalisation, de collecte et de valorisation des déchets, supervisé par un espace d'administration. L'application s'appuie sur une architecture client-serveur moderne : un client Flutter multiplateforme (Android, iOS, Web) communique avec Supabase, une plateforme Backend-as-a-Service fondée sur PostgreSQL et l'extension géospatiale PostGIS, qui assure l'authentification, le stockage des données, le stockage de fichiers et la sécurité d'accès via des politiques Row Level Security (RLS).

Le rapport détaillé successivement le contexte et la problématique locale, le cahier des charges fonctionnel et technique, l'analyse et la conception du système (architecture, modèle de données, diagrammes UML), la réalisation technique, la stratégie de tests, ainsi que le processus de déploiement continu via GitHub Actions. Il se conclut par un bilan critique du projet et des perspectives d'évolution, notamment l'amélioration du fonctionnement hors connexion et l'intégration de moyens de paiement mobiles réels, deux enjeux particulièrement pertinents dans le contexte camerounais.

Mots-clés : gestion des déchets, salubrité urbaine, Cameroun, Supabase, Backend-as-a-Service, PostGIS, géolocalisation, application mobile Flutter, Row Level Security, développement durable.

Abstract

Cameroonian cities face growing pressure related to urban solid waste management, a direct conséquence of rapid urbanization combined with a lack of digital tools for tracking public sanitation in real time. Reporting an illegal dumping site, coordinating collection teams, and monitoring waste treatment still largely rely on informal, poorly traceable channels that are difficult for municipal authorities to measure and act upon.

This report documents the design, development, and deployment of CleanCity, a cross-platform mobile and web application connecting three catégories of stakeholders -- waste-generating citizens, collectors, and sorting/processing centers -- around a complete cycle of reporting, geolocation, collection, and waste recovery, supervised through an administration space. The application relies on a modern client-server architecture: a cross-platform Flutter client (Android, iOS, Web) communicates with Supabase, a Backend-as-a-Service platform built on PostgreSQL and the PostGIS géospatial extension, which handles authentication, data storage, file storage, and access security through Row Level Security (RLS) policies.

The report successively details the local context and problem statement, the functional and technical requirements, the system analysis and design (architecture, data model, UML diagrams), the technical implémentation, the testing strategy, and the continuous deployment process via GitHub Actions. It concludes with a critical assessment of the project and perspectives for future development, in particular improved offline functionality and intégration of real mobile-money payment channels, two issues that are especially relevant in the Cameroonian context.

Keywords: waste management, urban sanitation, Cameroon, Supabase, Backend-as-a-Service, PostGIS, geolocation, Flutter mobile application, Row Level Security, sustainable development.

Liste des abreviations et acronymes

Sigle	Signification
API	Application Programming Interface (interface de programmation applicative)
BaaS	Backend-as-a-Service
CI/CD	Continuous Intégration / Continuous Deployment (intégration et déploiement continus)
CRUD	Create, Read, Update, Delete
GPS	Global Positioning System
JWT	JSON Web Token
MINEPDED	Ministère de l'Environnement, de la Protection de la Nature et du Développement Durable (Cameroun)
OTP	One-Time Password
PostGIS	Extension géospatiale du système de gestion de base de données PostgreSQL
REST	Representational State Transfer
RLS	Row Level Security (sécurité au niveau des lignes)
RPC	Remote Procedure Call
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
UML	Unified Modeling Language
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
XAF	Franc CFA (BEAC), devise utilisée en Afrique centrale

Liste des figures

- Figure 1 -- Architecture globale du système CleanCity (vue contexte et conteneurs)
- Figure 2 -- Diagramme de cas d'utilisation
- Figure 3 -- Diagramme de classes (modèle de données applicatif)
- Figure 4 -- Diagramme entite-relation de la base de données
- Figure 5 -- Diagramme de séquence : cycle de vie d'un signallement de déchet
- Figure 6 -- Arborescence du code source de l'application Flutter
- Figure 7 -- Pipeline d'intégration et de déploiement continu (CI/CD)

Liste des tableaux

- Tableau 1 -- Comparaison entre le signallement traditionnel et le signallement via CleanCity
- Tableau 2 -- Acteurs du système et besoins associés
- Tableau 3 -- Spécifications fonctionnelles retenues
- Tableau 4 -- Spécifications non fonctionnelles retenues
- Tableau 5 -- Comparaison entre une architecture Backend-as-a-Service et un backend développé sur mesure
- Tableau 6 -- Dictionnaire de données des tables principales
- Tableau 7 -- Synthèse des politiques de sécurité (Row Level Security) par table
- Tableau 8 -- Principales dépendances techniques du projet
- Tableau 9 -- Scénarios de test fonctionnels et résultats
- Tableau 10 -- Étapes du pipeline CI/CD GitHub Actions

Table des matieres

Dedicace	3
Remerciements	4
Résumé.....	5
Abstract	6
Liste des abreviations et acronymes	7
Liste des figures.....	9
Liste des tableaux.....	9
Table des matieres.....	10
Introduction générale.....	13
Chapitre 1 -- Contexte du projet et problématique locale	14
1.1 Contexte géographique et socio-urbain du Cameroun	14
1.2 État des lieux de la gestion des déchets dans les villes camerounaises.....	14
1.3 Problématique : l'absence de suivi numérique en temps réel.....	15
1.4 Objectifs du projet CleanCity.....	15
1.5 Vision et valeurs du projet.....	16
Chapitre 2 -- Cahier des charges fonctionnel et technique	18
2.1 Expression des besoins.....	18
2.2 Spécifications fonctionnelles.....	19
2.3 Spécifications non fonctionnelles.....	21
2.4 Contraintes et périmètre du projet	22
Chapitre 3 -- Analyse et conception	23
3.1 Architecture globale du système	23
3.2 Modélisation des données et choix de Supabase	23
3.3 Structure de la base de données.....	24
3.4 Sécurité des données : Row Level Security	26
3.5 Diagrammes UML complémentaires.....	28

Chapitre 4 -- Technologies et réalisation	29
4.1 Choix de la stack technique	29
4.2 Backend-as-a-Service : Supabase en pratique.....	30
4.3 Intégration cartographique	30
4.4 Notifications push.....	31
4.5 Gestion de version avec GitHub	31
4.6 Organisation du code source.....	31
Chapitre 5 -- Phase de tests.....	33
5.1 Stratégie de tests.....	33
5.2 Scénarios de test et résultats	33
5.3 Gestion des anomalies et correctifs	35
Chapitre 6 -- Déploiement et maintenance.....	36
6.1 Pipeline d'intégration et de déploiement continu	36
6.2 Hébergement et infrastructure	37
6.3 Sécurisation de l'application	37
6.4 Perspectives de maintenance et évolution technique	38
Chapitre 7 -- Bilan, compétences acquises et perspectives	39
7.1 Bilan technique du projet.....	39
7.2 Limités identifiées	39
7.3 Compétences acquises	39
7.4 Perspectives d'évolution	40
Conclusion générale	40
Bibliographie	42
Annexes	43
Annexe A -- Extrait du schema de base de données (schema.sql)	43
Annexe B -- Extrait du pipeline CI/CD (flutter_ci_cd.yml)	43
Annexe C -- Captures d'écran	43

[Champ Word -- clic droit puis 'Mettre a jour les champs' pour actualiser la pagination avant impression.]

Introduction générale

La croissance urbaine que connaissent les principales villes camerounaises -- Douala, Yaounde, Bafoussam ou Garoua -- s'accompagne d'une production de déchets solides en hausse constante, que les dispositifs municipaux de collecte peinent à absorber intégralement. Dans de nombreux quartiers, en particulier les zones à habitat spontané et les périphéries en expansion rapide, l'absence de points de collecte réguliers favorise l'apparition de décharges sauvages, avec des conséquences directes sur la salubrité publique, l'écoulement des eaux pluviales et la santé des populations.

Face à ce constat, le numérique constitue un levier de plus en plus mobilisé à travers le monde pour améliorer la gestion urbaine : géolocalisation des signalements citoyens, suivi en temps réel des équipes de terrain, tableaux de bord pour les décideurs municipaux. C'est dans cette perspective que s'inscrit le projet CleanCity, une application mobile et web dont l'objectif est de digitaliser et de fluidifier la chaîne qui va du signalement d'un déchet par un citoyen jusqu'à sa collecte, son traitement et sa valorisation par un centre de tri, en passant par la coordination des collecteurs de terrain.

Le présent mémoire a pour objectif de rendre compte de manière structurée de l'ensemble de la démarche d'ingénierie logicielle ayant conduit à la réalisation de CleanCity, depuis l'analyse du contexte et l'expression des besoins jusqu'au déploiement de l'application, en passant par la conception architecturale, le choix des technologies et la validation par les tests. Il est organisé en sept chapitres.

Le premier chapitre présente le contexte géographique et la problématique locale qui motivent le projet. Le deuxième chapitre expose le cahier des charges fonctionnel et technique, en identifiant les acteurs du système et leurs besoins. Le troisième chapitre est consacré à l'analyse et à la conception du système : architecture globale, modélisation des données et diagrammes UML. Le quatrième chapitre détaille la réalisation technique et les choix technologiques opérés. Le cinquième chapitre présente la stratégie de test adoptée et ses résultats. Le sixième chapitre décrit le processus de déploiement et les mesures de sécurisation mises en place. Enfin, le septième chapitre dresse un bilan critique du projet et ouvre des perspectives d'évolution, avant une conclusion générale.

Chapitre 1 -- Contexte du projet et problématique locale

1.1 Contexte géographique et socio-urbain du Cameroun

Le Cameroun, souvent qualifié d'"Afrique en miniature" en raison de sa diversité géographique et culturelle, connaît depuis plusieurs décennies une urbanisation rapide et largement non planifiée. Les deux principales métropoles, Douala (capitale économique) et Yaounde (capitale politique), concentrent une part croissante de la population nationale, à laquelle s'ajoutent des villes secondaires en expansion telle que Bafoussam, Bamenda, Garoua ou Maroua. Cette dynamique urbaine, si elle témoigne d'un dynamisme économique et démographique réel, se heurte à des infrastructures de base -- voirie, assainissement, gestion des déchets -- dont le développement ne suit pas toujours le même rythme.

Dans ce contexte, la gestion des déchets solides urbains constitue un défi structurel partagé par de nombreuses villes d'Afrique subsaharienne : les circuits de collecte formels, souvent gérés par des prestataires municipaux ou des délégataires de service public, coexistent avec des pratiques informelles de dépôt dans des espaces non aménagés (terrains vagues, abords de cours d'eau, bas-fonds). Ce phénomène est documenté de manière récurrente par les organismes internationaux tels que l'ONU-Habitat ou la Banque mondiale dans leurs rapports sur la gestion des déchets solides en Afrique subsaharienne, qui soulignent le lien entre insuffisance de la collecte, déficit d'infrastructures et risques sanitaires et environnementaux accrus (inondations liées à l'obstruction des drains, prolifération de vecteurs de maladies).

[Note méthodologique : l'étudiant est invité à compléter ce paragraphe avec des données chiffrées actualisées et sourcées, par exemple issues de l'Institut National de la Statistique du Cameroun, du Ministère de l'Environnement, de la Protection de la Nature et du Développement Durable (MINEPDED), ou de rapports d'organismes internationaux spécifiques à la ou aux villes étudiées dans le cadre du projet.]

1.2 État des lieux de la gestion des déchets dans les villes camerounaises

La chaîne de gestion des déchets, dans son fonctionnement traditionnel, repose généralement sur les étapes suivantes : dépôt du déchet par le ménage ou l'entreprise génératrice, collecte par un opérateur (souvent selon un circuit et un calendrier fixes), transport vers un site de transfert ou de traitement, puis enfouissement, incinération ou valorisation. Ce modèle, largement linéaire, présente plusieurs limites dans le contexte camerounais : couverture géographique incomplète de la collecte régulière, absence de canal simple permettant à un citoyen de signaler un point de décharge sauvage en dehors

des circuits prévus, et manque de visibilité, pour les autorités comme pour les opérateurs, sur l'état réel du territoire à un instant donné.

À cela s'ajoute une dimension économique et sociale importante : une partie non négligeable de la collecte et du tri s'effectue de manière informelle, par des acteurs individuels qui récupèrent des matériaux valorisables (plastique, métal, papier) sans cadre structure de rémunération ni de traçabilité. Cette économie informelle, bien que porteuse d'un potentiel de valorisation réel, souffre d'un manque de reconnaissance, de sécurité et de visibilité qui limite son impact positif sur la salubrité urbaine.

1.3 Problématique : l'absence de suivi numérique en temps réel

La problématique centrale de ce projet peut se formuler ainsi : comment permettre à un citoyen de signaler rapidement et précisément un problème de salubrité (déchet non collecté, décharge sauvage), tout en offrant aux acteurs de la collecte et du traitement -- collecteurs de terrain, centres de tri, autorités -- une visibilité en temps réel sur l'ensemble des signalements, leur localisation et leur statut de traitement ?

En l'absence d'un tel dispositif numérique, plusieurs conséquences sont observées : les signalements informels (appel téléphonique, réseaux sociaux, bouche-à-oreille) se perdent ou ne sont pas centralisés ; les collecteurs ne disposent pas d'une vue d'ensemble des zones prioritaires ; les centres de tri ne peuvent pas anticiper les volumes et types de déchets à recevoir ; enfin, il n'existe aucune traçabilité permettant de mesurer objectivement la performance du système de collecte (délai moyen de traitement, volumes collectés par zone, taux de valorisation).

C'est cette absence d'outil numérique de suivi, de géolocalisation et de coordination en temps réel entre les différents acteurs de la chaîne de gestion des déchets qui constitue le cœur de la problématique traitée par le projet CleanCity.

1.4 Objectifs du projet CleanCity

Le projet CleanCity poursuit un objectif général : concevoir et développer une plateforme numérique, accessible via mobile (Android, iOS) et via le web, permettant de digitaliser l'ensemble du cycle de vie d'un signalement de déchet, depuis sa création par un citoyen jusqu'à sa valorisation par un centre de tri, en passant par sa prise en charge par un collecteur.

- Permettre à tout citoyen de signaler, en quelques étapes, un déchet ou une décharge sauvage en y associant une localisation GPS précise et des photographies.
- Offrir aux collecteurs un tableau de bord des missions disponibles, localisées sur une carte, avec la possibilité d'accepter, de suivre et de clôturer une mission de collecte.

- Permettre aux centres de tri de confirmer la reception des déchets collectes, d'en saisir le poids réel et de déclencher, le cas echeant, une rémunération liée a la valorisation (système de points écologiques convertibles).
- Fournir aux administrateurs de la plateforme une vue d'ensemble des utilisateurs, des signalements et des demandes de paiement, avec les outils de supervision nécessaires.
- Favoriser, a travers un système de points écologiques et un mécanisme de retrait par mobile money, une incitation économique a la collecte et au tri des déchets valorisables.

1.5 Vision et valeurs du projet

CleanCity s'inscrit dans une démarche de développement durable resumee par sa signature : « Collect - Recover - Preserve » (Collecter - Valoriser - Preserver). Cette formule traduit une vision en trois temps : collecter les déchets la ou ils se trouvent grâce a la participation citoyenne, valoriser les matieres récupérables au sein de centres de tri partenaires, et préserver ainsi l'environnement urbain et la santé publique.

Au-dela de l'aspect technique, le projet porte une ambition sociale et environnementale : rendre visible un problème souvent subi en silence par les populations, structurer une économie de la collecte aujourd'hui largement informelle, et outiller les acteurs locaux -- citoyens, collecteurs, centres de tri, autorités -- d'un langage commun autour de la donnée géolocalisée.

Critere	Signalement traditionnel	Signalement via CleanCity
Canal de signalement	Appel téléphonique, bouche-a-oreille, réseaux sociaux	Application mobile / web dediee
Localisation	Description approximative, non standardisee	Coordonnées GPS précises rattachees a une adresse
Traçabilité du statut	Aucune visibilite pour le citoyen apres signalement	Statuts suivis en temps réel (en attente, accepte, collecte, livre)
Preuve visuelle	Rarement disponible	Photographies jointes au signalement
Coordination des collecteurs	Manuelle, non centralisee	Missions visibles sur une carte, attribuees et suivies

Mesure de la performance	Non mesurable objectivement	Historique exploitable (délais, volumes, statuts)
Incitation a la collecte	Absente ou informelle	Points écologiques et retrait par mobile money

Tableau 1 -- Comparaison entre le signalement traditionnel et le signalement via CleanCity

Chapitre 2 -- Cahier des charges fonctionnel et technique

2.1 Expression des besoins

L'analyse des besoins a été conduite en identifiant quatre catégories d'acteurs directement impliqués dans l'usage de la plateforme, chacune correspondant à un rôle applicatif distinct dans le système CleanCity (champ rôle de la table users : generator, collector, center, admin).

Acteur	Rôle applicatif	Besoins principaux
Citoyen / générateur de déchets	generator	Signaler rapidement un déchet avec photo et localisation ; suivre le statut de ses signalements ; être notifié de la prise en charge ; contacter le collecteur.
Collecteur	collector	Consulter les missions disponibles sur une carte ; accepter une mission ; marquer la collecte et la livraison ; communiquer avec le générateur ; suivre sa rémunération.
Centre de tri	center	Recevoir les livraisons associées à son centre ; confirmer la réception et saisir le poids réel ; enregistrer un événement de traitement ; déclencher le crédit de points au collecteur.
Administrateur	admin	Superviser l'ensemble des utilisateurs et des signalements ; valider ou rejeter les demandes de retrait (payout) ; disposer d'une vue statistique globale de la plateforme.

Tableau 2 -- Acteurs du système et besoins associés

Cette structuration en quatre rôles distincts, portée au niveau de la base de données par le type énuméré `user_role`, permet d'appliquer des règles d'accès différenciées (voir 3.4) et de proposer, au sein d'une seule et même application, des parcours utilisateurs adaptés à chaque profil.

2.2 Spécifications fonctionnelles

Les spécifications fonctionnelles retenues couvrent l'ensemble du cycle de vie d'un signalement de déchet, de sa création à sa valorisation, ainsi que les fonctionnalités transverses (authentification, messagerie, notifications).

Référence	Fonctionnalité	Description sommaire
FR-01	Authentification multi-canal	Inscription et connexion par email/mot de passe, par compte Google (OAuth), ou par numéro de téléphone (code OTP).
FR-02	Sélection et verrouillage du rôle	Choix du rôle applicatif à la première connexion ; modification ultérieure réservée à l'administrateur (règle métier <code>enforce_role_change</code>).
FR-03	Création d'un signalement géolocalisé	Saisie du type de déchet, de la quantité estimée, de l'adresse et d'un créneau horaire, avec capture automatique des coordonnées GPS.
FR-04	Ajout de photographies	Prise ou sélection d'une photo jointe au signalement, hébergée dans le stockage Supabase.
FR-05	Visualisation cartographique	Affichage des signalements et des missions sur une carte interactive (fond de carte Mapbox).
FR-06	Recherche géographique de proximité	Recherche des signalements en attente dans un rayon donné autour d'une position, via la fonction PostGIS <code>nearby_waste_requests</code> .

FR-07	Prise en charge d'une mission	Acceptation d'un signalement par un collecteur, avec verrouillage pour éviter une double affectation.
FR-08	Suivi du statut de collecte	Mise a jour du statut du signalement (en attente, accepte, collecte, livre, annule) visible par toutes les parties prenantes.
FR-09	Reception et pesee au centre de tri	Confirmation de reception, saisie du poids réel et enregistrement d'un événement de traitement.
FR-10	Système de points écologiques	Credit automatique de points au collecteur en fonction du type et du poids de déchet valorisé.
FR-11	Demande de retrait Mobile Money	Conversion des points en un montant en francs CFA (XAF) et demande de retrait via un opérateur de mobile money, validée par un administrateur.
FR-12	Messagerie instantanee	Échange de messages entre générateur et collecteur, dans le cadre d'un signalement ou en conversation directe.
FR-13	Notifications push	Notification des utilisateurs concernés a chaque changement d'état significatif (nouvelle mission, mission acceptee, paiement reçu).
FR-14	Espace d'administration	Gestion des utilisateurs, supervision des signalements et validation des demandes de retrait par l'administrateur.

Tableau 3 -- Spécifications fonctionnelles retenues

2.3 Spécifications non fonctionnelles

Au-delà des fonctionnalités métier, plusieurs exigences non fonctionnelles ont guidé les choix d'architecture et d'implémentation, en tenant compte des contraintes spécifiques du contexte camerounais, notamment la variabilité de la qualité de connexion internet en dehors des grands centres urbains.

Catégorie	Exigence	Justification
Performance	Temps de réponse acceptable sur les opérations courantes (chargement de la carte, liste des missions) même en connexion mobile 3G/4G limitée.	Une partie significative des utilisateurs accède à l'application via des réseaux mobiles dont le débit est variable selon les zones.
Sécurité des données	Isolation stricte des données par utilisateur et par rôle via des politiques Row Level Security au niveau de la base de données.	Éviter qu'un utilisateur puisse consulter ou modifier des données ne le concernant pas, indépendamment de la couche applicative.
Accessibilité / faible connectivité	Conception d'interfaces légères, limitation du poids des images transmises, gestion explicite des erreurs réseau.	Certaines zones peri-urbaines et rurales camerounaises présentent une connectivité internet intermittente.
Ergonomie mobile	Interface adaptée à une utilisation tactile, parcours simplifiés pour un public non nécessairement familier avec les outils numériques.	Le succès de l'adoption citoyenne dépend directement de la simplicité du parcours de signalement.
Multiplateforme	Une base de code unique déployable sur Android, iOS et Web.	Réduire les coûts de développement et de maintenance tout en maximisant la couverture d'utilisateurs potentiels.
Disponibilité	Recours à une infrastructure cloud managée (Supabase) offrant une disponibilité élevée sans gestion d'infrastructure dédiée.	Le porteur de projet, dans un contexte de mémoire de fin d'études, ne dispose pas d'une équipe d'exploitation dédiée.

Localisation linguistique	Interface et messages en français, avec un champ preferred_language pour une extension future multilingue.	Le français est la langue d'usage principale du public cible dans les zones concernées.
---------------------------	--	---

Tableau 4 -- Spécifications non fonctionnelles retenues

2.4 Contraintes et périmètre du projet

Le périmètre du projet a été volontairement circonscrit afin de rester compatible avec le cadre temporel d'un mémoire de licence. Sont ainsi inclus dans le périmètre : le cycle complet signalement-collecte-traitement, l'authentification multi-canal, la cartographie, la messagerie et le système de points écologiques. En sont exclus, à ce stade, l'intégration réelle d'une passerelle de paiement mobile money (le mécanisme actuel repose sur une demande de retrait validée manuellement par un administrateur) et le mode de fonctionnement entièrement hors connexion, qui constituent des pistes d'évolution présentées au chapitre 7.

Chapitre 3 -- Analyse et conception

3.1 Architecture globale du système

L'architecture de CleanCity repose sur un modèle client-serveur classique, dans lequel le client -- l'application Flutter -- concentre l'ensemble de la logique de présentation et de navigation, tandis que le serveur -- la plateforme Supabase -- assume les responsabilités de persistance, d'authentification, de stockage de fichiers et de sécurité d'accès. Cette répartition permet de déployer une base de code Dart unique sur trois cibles (Android, iOS, Web) sans dupliquer la logique métier cote serveur.

Le client communique avec Supabase via une API REST auto-générée (PostgREST) au-dessus de la base PostgreSQL, ainsi que via les SDK dédiés à l'authentification et au stockage de fichiers. Deux services tiers complètent l'architecture : Mapbox, pour la fourniture des fonds de carte et des tuiles cartographiques affichées dans le composant CleanCityMapView, et OneSignal, pour l'envoi des notifications push aux utilisateurs concernés par un changement de statut.

Emplacement visuel suggère -- Figure 1

Diagramme d'architecture (vue C4 - niveau Contexte et niveau Conteneurs) : au centre, l'application CleanCity (client Flutter mobile + web) ; autour, les acteurs (générateur, collecteur, centre de tri, administrateur) et les systèmes externes (Supabase : Auth, PostgreSQL, Storage, Realtime ; Mapbox ; OneSignal ; GitHub Actions / Codemagic pour le CI/CD). Source Mermaid disponible dans docs/DIAGRAMMES_UML_C4.md (sections 4.1 et 4.2), à exporter en image via mermaid.live avant insertion.

3.2 Modélisation des données et choix de Supabase

Le choix d'une architecture de type Backend-as-a-Service (BaaS), en l'occurrence Supabase, s'est imposé au regard des contraintes de temps et de ressources propres à un projet de fin d'études. Supabase fournit, au-dessus d'une base PostgreSQL standard, un ensemble de services intégrés -- authentification (email, OAuth, OTP téléphonique), API REST auto-générée, stockage de fichiers avec règles d'accès, souscriptions en temps réel -- qui auraient nécessité, dans une approche de backend développé sur mesure, un investissement de développement et d'exploitation nettement plus important.

Un autre argument déterminant en faveur de Supabase est la disponibilité native de l'extension PostGIS, qui permet de stocker des coordonnées géographiques sous forme de colonnes de type geography et d'exécuter directement, cote base de données, des requêtes de proximité spatiale performantes (fonction ST_DWithin, tri par distance via l'opérateur <->). Cette approche évite de

dupliquer une logique de calcul géographique côté client et garantit la cohérence des résultats quelle que soit la plateforme d'accès (mobile ou web).

Critere	Backend-as-a-Service (Supabase)	Backend développé sur mesure
Délai de mise en oeuvre	Court : authentification, API et stockage disponibles dès la création du projet	Long : nécessite de développer et de sécuriser chaque brique (auth, API, stockage)
Coût d'infrastructure	Mutualisé, adapté à un usage naissant (offre gratuite / faible coût au démarrage)	Nécessite un serveur ou une infrastructure cloud dédiée dès le départ
Sécurité d'accès	Gérée au plus près de la donnée via des politiques Row Level Security	A implémenter intégralement au niveau applicatif ou via un middleware dédié
Fonctions géospatiales	PostGIS intégré nativement à PostgreSQL	Nécessite l'installation et la maintenance d'une extension ou d'un service dédié
Maintenance et exploitation	Assurées par le fournisseur (mises à jour, sauvegardes, disponibilité)	A la charge intégrale de l'équipe projet
Flexibilité / personnalisation	Bonne, via fonctions SQL et RPC personnalisées, mais dépendante des capacités de la plateforme	Totale, au prix d'un effort de développement plus conséquent

Tableau 5 -- Comparaison entre une architecture Backend-as-a-Service et un backend développé sur mesure

3.3 Structure de la base de données

Le schéma de la base de données, défini dans le fichier lib/supabase/schema.sql, s'articule autour de onze tables principales couvrant l'ensemble du cycle fonctionnel de l'application : gestion des profils utilisateurs, des adresses géolocalisées, des signalements de déchets, des missions de collecte, des événements de traitement, des transactions de points écologiques, des demandes de retrait et de la messagerie.

Table	Champs clés	Rôle dans le système
users	id, rôle, full_name, phone_e164, role_confirmed_at	Profil applicatif de chaque utilisateur, lié a auth.users ; porte le rôle (generator, collector, center, admin).
addresses	id, user_id, latitude, longitude, location (geography)	Adresses géolocalisées ; la colonne location est calculée automatiquement à partir de latitude/longitude via PostGIS.
waste_requests	id, generator_id, address_id, center_id, waste_type, status	Signalement de déchet : entité centrale du système, avec son statut (pending, accepted, collected, delivered, cancelled).
pickups	request_id, collector_id, accepted_at, collected_at, delivered_at	Mission de collecte associée à un signalement, portée par un collecteur.
waste_request_photos	id, request_id, uploaded_by, url	Photographies associées à un signalement, stockées dans Supabase Storage.
processing_events	id, request_id, center_id, weighed_kg, accepted	Événement de pesée et de validation d'un signalement par un centre de tri.
eco_transactions	id, user_id, request_id, points, reason	Historique des points écologiques crédités à un utilisateur.
payout_requests	id, user_id, provider, phone, amount_xaf, status	Demande de retrait des points convertis en francs CFA via un opérateur de mobile money.
chat_threads chat_thread_members chat_messages	/ id, kind, request_id / thread_id, / user_id / sender_id, body	Messagerie liée à un signalement (kind = request) ou conversation directe (kind = direct).

Tableau 6 -- Dictionnaire de données des tables principales

Emplacement visuel suggere -- Figure 3 et Figure 4

Diagramme de classes du modèle applicatif (AppUser, WasteRequest, Pickup, Address, WasteRequestPhoto, ProcessingEvent, EcoTransaction, PayoutRequest et leurs services associés) et diagramme entite-relation de la base de données, tous deux disponibles au format Mermaid dans docs/DIAGRAMMES_UML_C4.md (sections 2 et 5).

Un point de conception notable concerné la colonne location de la table addresses, définie comme une colonne générée (generated always as ... stored) : elle est recalculée automatiquement par PostgreSQL à chaque insertion ou mise à jour des colonnes latitude et longitude, garantissant qu'aucune incohérence ne puisse exister entre les coordonnées brutes et leur représentation géographique. Cette colonne est indexée au moyen d'un index spatial GiST (addresses_location_gix), condition nécessaire à la performance des requêtes de proximité.

La recherche de signalements à proximité d'une position donnée est encapsulée dans une fonction serveur dédiée, nearby_waste_requests(lat, lng, radius_meters), qui combine un filtre sur le statut du signalement (pending), une clause de distance ST_DWithin exprimée en mètres, et un tri par distance croissante via l'opérateur PostGIS <->. Cette logique, exécutée directement dans la base de données, offre de meilleures performances qu'une implémentation côté client et demeure indépendante de la plateforme consommatrice (mobile ou web).

3.4 Sécurité des données : Row Level Security

La sécurité d'accès aux données constitue un axe de conception à part entière. Plutôt que de reposer uniquement sur des contrôles réalisés côté application -- par nature contournables si un utilisateur malveillant interagit directement avec l'API -- CleanCity applique le principe de défense en profondeur en activant les politiques Row Level Security (RLS) de PostgreSQL sur l'intégralité des tables applicatives. Chaque requête émise, y compris via l'API REST auto-générée, est ainsi filtrée au niveau du moteur de base de données en fonction de l'identité de l'utilisateur authentifié (auth.uid()).

Deux fonctions utilitaires, exécutées en mode SECURITY DEFINER pour éviter les recursions de politiques, structurent l'ensemble des règles d'accès : current_user_role(), qui retourne le rôle de l'utilisateur courant, et is_request_participant(request_id), qui détermine si l'utilisateur courant est partie prenante d'un signalement donné (générateur, collecteur assigné ou centre destinataire). Un déclencheur (enforce_role_change) empêche par ailleurs un utilisateur de modifier lui-même son rôle applicatif une fois celui-ci confirmé, cette opération étant réservée à l'administrateur.

Table	Politique principale	Règle appliquée
users	users_update_self_or_admin	Un utilisateur ne peut modifier que son propre profil, sauf s'il est administrateur.
waste_requests	waste_requests_update_owner_or_agent	Modification réservée au générateur, au collecteur assigné, ou à un centre/administrateur.
pickups	pickups_select_participant	Lecture réservée au collecteur, au centre concerné, au générateur du signalement, ou à l'administrateur.
waste_request_photos	photos_select_participant	Lecture et dépôt réservés aux participants du signalement (via is_request_participant).
eco_transactions	eco_transactions_insert_agent	Seul un centre de tri ou un administrateur peut créditer des points à un utilisateur.
payout_requests	payout_requests_update_admin	Seul un administrateur peut faire évoluer le statut d'une demande de retrait.
chat_messages	chat_messages_insert_member	Un message ne peut être envoyé que par un membre du fil de discussion concerné.
storage.objects (request_photos)	storage_request_photos_participant_read/write	Accès aux photos d'un signalement réservé aux participants de ce signalement.

Tableau 7 -- Synthèse des politiques de sécurité (Row Level Security) par table

3.5 Diagrammes UML complémentaires

Afin de documenter le comportement dynamique du système, deux catégories de diagrammes complémentaires ont été produites : un diagramme de cas d'utilisation, formalisant les interactions entre les quatre acteurs identifiés et les fonctionnalités du système, et un ensemble de diagrammes de séquence, illustrant le cheminement d'un signalement de sa création à sa valorisation.

Emplacement visuel suggere -- Figure 2

Diagramme de cas d'utilisation regroupant les acteurs Générateur, Collecteur, Centre de tri et Administrateur autour des cas d'utilisation Authentification, Signalements, Missions et Administration (source Mermaid : docs/DIAGRAMMES_UML_C4.md, section 1).

Emplacement visuel suggere -- Figure 5

Diagramme de séquence du cycle de vie complet d'un signalement : (1) création par le générateur avec capture GPS et upload photo, (2) acceptation de la mission par un collecteur, (3) marquage collecte puis livraison au centre, (4) réception, pesée et crédit de points par le centre de tri, (5) demande de retrait Mobile Money validée par l'administrateur (source Mermaid : docs/DIAGRAMMES_UML_C4.md, section 3).

Le passage d'un signalement par les statuts pending, accepted, collected puis delivered materialise ainsi une machine à états simple mais rigoureuse, dont chaque transition est contrôlée à la fois par la logique applicative (écrans et services Flutter) et par les politiques RLS présentées en 3.4, garantissant qu'aucune transition ne puisse être déclenchée par un acteur non habilité.

Chapitre 4 -- Technologies et réalisation

4.1 Choix de la stack technique

Le choix technologique central du projet est celui du framework Flutter (SDK Dart ^3.6.0), retenu pour sa capacité à produire, à partir d'une base de code unique écrite en Dart, des applications natives Android et iOS ainsi qu'une application Web, sans sacrifier les performances ni la cohérence visuelle entre plateformes. Ce choix répond directement à une exigence non fonctionnelle du cahier des charges (2.3) : maximiser la couverture d'utilisateurs -- citoyens équipés de smartphones Android, collecteurs et centres pouvant préférer un usage web sur poste fixe -- avec un effort de développement mutualisé.

La navigation applicative repose sur le package `go_router`, qui structure les différents parcours (authentification, tableau de bord générateur, collecteur, centre, administration) sous forme de routes déclaratives, facilitant la gestion des redirections liées au rôle de l'utilisateur connecté. La gestion d'état transverse (theme clair/sombre notamment) s'appuie sur le package `provider`, dans une architecture volontairement simple au regard du périmètre fonctionnel du projet.

Dépendance	Rôle dans l'application
<code>supabase_flutter</code>	Client officiel Supabase : authentification, requêtes PostgreSQL, stockage, souscriptions temps réel.
<code>go_router</code>	Navigation déclarative et gestion des routes selon le rôle de l'utilisateur.
<code>provider</code>	Gestion d'état transverse (theme, état d'authentification).
<code>flutter_map</code> + <code>latlong2</code>	Affichage de cartes interactives et de marqueurs géolocalisés.
<code>geolocator</code>	Accès à la position GPS de l'appareil.
<code>image_picker</code>	Sélection ou capture de photographies pour les signalements.
<code>mime</code>	Détection du type MIME des fichiers avant upload vers le stockage.

onesignal_flutter	Intégration du service de notifications push OneSignal.
google_fonts	Typographies cohérentes avec la charte graphique de l'application.
http	Requetes HTTP complémentaires vers des services externes.

Tableau 8 -- Principales dépendances techniques du projet

4.2 Backend-as-a-Service : Supabase en pratique

L'intégration de Supabase s'articule autour de quatre briques principales, initialisées au démarrage de l'application dans `lib/supabase/supabase_config.dart` puis consommées par des services dédiés (`AppUserService`, `WasteRequestService`, `PayoutRequestService`, `ChatService`). L'authentification est gérée par le module Supabase Auth, qui prend en charge l'inscription par email et mot de passe, la connexion via un fournisseur OAuth Google, et la vérification par code à usage unique (OTP) envoyée par SMS pour les connexions par numéro de téléphone.

Le stockage des fichiers repose sur deux buckets distincts : `user_uploads`, public en lecture, dédié aux avatars des utilisateurs, et `request_photos`, privé et soumis à une politique d'accès restreignant la lecture et l'écriture aux seuls participants du signalement concerné (voir 3.4). Cette séparation traduit, au niveau du stockage de fichiers, la même logique de moindre privilège que celle appliquée aux tables de la base de données.

Enfin, la fonction RPC `nearby_waste_requests`, appelée depuis le service applicatif de recherche géographique, illustre l'usage de fonctions SQL personnalisées exposées via l'API Supabase : plutôt que de rapatrier l'ensemble des signalements pour les filtrer côté client, l'application délègue le calcul de proximité géographique à la base de données elle-même, ce qui réduit à la fois la quantité de données transférées -- un enjeu important en contexte de connectivité limitée -- et le temps de traitement.

4.3 Intégration cartographique

La visualisation cartographique, portée par le composant partage `CleanCityMapView`, s'appuie sur le package `flutter_map` associé à des tuiles vectorielles fournies par l'API Static Tiles de Mapbox. Ce choix fait suite à une première tentative d'intégration du SDK natif `mapbox_maps_flutter`, dont la compatibilité se limitait aux plateformes Android et iOS ; l'adoption de `flutter_map`, alimentée par les tuiles Mapbox via une simple URL de tuiles paramétrée par un jeton d'accès

(MAPBOX_ACCESS_TOKEN), a permis d'obtenir un rendu cartographique cohérent sur les trois plateformes cibles, y compris le Web.

Un mécanisme de repli (fallback) a été prévu : en l'absence de jeton Mapbox configure, l'application bascule automatiquement sur les tuiles publiques d'OpenStreetMap, garantissant qu'une carte reste toujours affichée, y compris en environnement de développement ou de démonstration ne disposant pas encore d'une clé Mapbox validée.

Emplacement visuel suggère -- Figure 6

Capture d'écran de l'écran carte (CleanCityMapView) affichant les signalements en attente sous forme de marqueurs colorés, à insérer depuis l'application en cours d'exécution.

4.4 Notifications push

La notification des utilisateurs lors des changements d'état significatifs (nouvelle mission disponible, mission acceptée, paiement reçu) est assurée par le service PushNotificationService, qui encapsule l'intégration du SDK OneSignal. Ce choix permet de bénéficier d'une gestion centralisée des jetons d'appareils et des segments d'audience, sans avoir à développer une infrastructure de notification dédiée (serveur de push, gestion des certificats Apple Push Notification service, etc.).

4.5 Gestion de version avec GitHub

Le développement du projet est versionné sur GitHub, avec une branche principale (main) servant de référence stable et de base au déclenchement du pipeline d'intégration continue. L'historique de commits du projet illustre une progression itérative caractéristique d'un développement de mémoire de fin d'études : mise en place initiale du projet et des pipelines CI/CD, correctifs de build (par exemple la mise à jour du package google_fonts), puis une refonte majeure du backend autour de Supabase et PostGIS accompagnée de la migration de la couche cartographique vers Mapbox.

Cette organisation, bien que simplifiée par rapport à un flux de travail Gitflow complet (absence de branches de fonctionnalités systématiques dans l'historique observé), a permis de maintenir à tout moment une branche principale compilable, condition nécessaire au bon fonctionnement du pipeline de build automatisé décrit au chapitre 6.

4.6 Organisation du code source

Le code source de l'application est organisé en modules correspondant aux rôles applicatifs et aux domaines fonctionnels, favorisant la lisibilité et la maintenabilité du projet malgré l'absence d'une architecture en couches strictement découpée (par exemple de type Clean Architecture).

Emplacement visuel suggère -- Figure 7

Arborescence du dossier lib/ : auth/, generator/, collector/, center/, admin/, chat/ (écrans par rôle et par domaine) ; models/ (AppUser, WasteRequest, PayoutRequest) ; services/ (AppUserService, WasteRequestService, ChatService, MediaUploadService, PushNotificationService, MapsService) ; components/ (widgets partagés dont CleanCityMapView) ; supabase/ (configuration et schema.sql) ; nav.dart (configuration go_router) ; main.dart (point d'entrée).

Cette organisation par rôle (auth, generator, collector, center, admin, chat) plutôt que par couche technique présente l'avantage de regrouper, pour chaque profil d'utilisateur, l'ensemble des écrans qui le concernent, facilitant la compréhension du parcours utilisateur correspondant lors de la maintenance ou de l'ajout de nouvelles fonctionnalités.

Chapitre 5 -- Phase de tests

5.1 Stratégie de tests

La validation de CleanCity a été conduite selon trois niveaux complémentaires. Les tests unitaires, écrits avec le package `flutter_test`, portent sur la logique pure des modèles de données (méthodes `fromJson/toJson` et `copyWith` des classes `AppUser`, `WasteRequest`, `PayoutRequest`) ainsi que sur les fonctions utilitaires ne dépendant pas d'un état d'interface (calculs liés aux points écologiques, formatage des libelles de rôle).

Les tests d'intégration visent à vérifier le comportement du système au niveau de la base de données, en particulier le respect effectif des politiques Row Level Security présentées en 3.4 : chaque règle d'accès a été contrôlée en tentant, avec des comptes de test associés à des rôles différents, des opérations autorisées et des opérations qui doivent être refusées (par exemple, un générateur tentant de modifier le signalement d'un autre utilisateur, ou un collecteur tentant de créditer des points sans être rattaché à un centre).

Enfin, les tests d'interface utilisateur ont été réalisés de manière manuelle et exploratoire, en parcourant l'ensemble des scénarios utilisateurs clés sur émulateur Android, sur navigateur Web et, dans la mesure du possible, sur un appareil physique, afin de valider à la fois le fonctionnement fonctionnel et l'ergonomie des parcours (voir 2.3, exigences non fonctionnelles d'ergonomie mobile).

5.2 Scénarios de test et résultats

Le tableau suivant synthétise un extrait représentatif des scénarios de test exécutés au cours du projet, couvrant les principales fonctionnalités et règles de sécurité du système.

ID	Scénario	Résultat attendu	Statut
T-01	Inscription par email puis sélection du rôle générateur	Compte créé, profil provisionné automatiquement (<code>handle_new_auth_user</code>), rôle enregistré	Réussi
T-02	Tentative de modification du rôle par l'utilisateur lui-même après confirmation	Opération rejetée (<code>ROLE_ALREADY_CONFIRMED</code>) sauf pour un administrateur	Réussi

T-03	Création d'un signalement avec photo et localisation GPS	Signalement, adresse et photo enregistrés ; statut initial pending	Réussi
T-04	Recherche de signalements a proximite (rayon 5 km)	Liste triee par distance croissante via nearby_waste_requests	Réussi
T-05	Acceptation d'une mission par deux collecteurs simultanement	Un seul enregistrement pickups accepte, le second rejete	Réussi
T-06	Marquage collecte puis livraison au centre selectionne	Statut du signalement mis a jour successivement en collected puis delivered	Réussi
T-07	Lecture d'un signalement par un utilisateur non participant	Accès en lecture aux photos et événements de traitement refuse par RLS	Réussi
T-08	Pesee et validation par le centre de tri	Enregistrement processing_events créé, credit de points écologiques au collecteur	Réussi
T-09	Credit de points en double pour un meme signalement	Contrainte unique (user_id, request_id, reason) empêchant le doublon	Réussi
T-10	Demande de retrait Mobile Money par un collecteur	Demande créée avec statut pending, visible par l'administrateur uniquement pour validation	Réussi
T-11	Envoi d'un message dans un fil de discussion sans en etre membre	Insertion refusee par la politique chat_messages_insert_member	Réussi

T-12	Chargement de la carte en connexion mobile dégradée (simulée)	Affichage progressif des tuiles, aucun blocage de l'interface	Réussi (avec latence observée, cf. 5.3)
T-13	Notification push lors de l'acceptation d'une mission	Le générateur concerné reçoit une notification	Partiellement réussi (délai variable selon la plateforme, cf. 5.3)

Tableau 9 -- Scénarios de test fonctionnels et résultats

5.3 Gestion des anomalies et correctifs

Le suivi des anomalies s'est appuyé sur les fonctionnalités natives de GitHub (issues et historique de commits), permettant de relier chaque correctif au problème identifié. Deux catégories d'anomalies ont été rencontrées de manière récurrente durant la phase de test.

La première concerné des erreurs de politique de sécurité au niveau de la base de données, notamment des cas de recursion infinie détectée dans les politiques RLS (erreur PostgreSQL 42P17) lorsqu'une politique sur la table `chat_thread_members` vérifiait sa propre appartenance par une sous-requête directe sur la même table. Le correctif a consisté à introduire une fonction dédiée en mode SECURITY DEFINER (`is_thread_member`), contournant ainsi la recursion tout en préservant la logique de contrôle d'accès.

La seconde catégorie porte sur des anomalies liées à la latence réseau en conditions de connectivité dégradée (scénario T-12 et T-13) : chargement lent des tuiles cartographiques et délai variable de réception des notifications push. Ces observations, bien que n'ayant pas empêché la validation fonctionnelle du système, ont été consignées comme axes d'amélioration prioritaires et sont reprises dans les perspectives d'évolution du chapitre 7.

Chapitre 6 -- Déploiement et maintenance

6.1 Pipeline d'intégration et de déploiement continu

Le projet met en oeuvre un pipeline d'intégration continue via GitHub Actions (fichier `.github/workflows/flutter_ci_cd.yml`), déclenche automatiquement à chaque push et à chaque pull request sur la branche main. Ce pipeline s'exécute sur un runner macOS, choix nécessaire pour permettre la compilation de la cible iOS aux côtés des cibles Android et Web.

Étape	Action réalisée
Checkout du code	Récupération du dépôt GitHub sur le runner d'exécution
Configuration Java 17	Installation de la distribution Zulu, requise par les versions récentes de Gradle/Android
Configuration Flutter (canal stable)	Installation du SDK Flutter via l'action subosito/flutter-action
Installation des dépendances	Exécution de flutter pub get
Build Android	Compilation de l'APK de release (flutter build apk --release)
Build iOS	Compilation de l'application iOS sans signature (flutter build ios --release --no-codesign)
Build Web	Compilation de la version web de production (flutter build web --release)
Publication des artefacts	Dépôt des trois builds (APK, application iOS, build web) comme artefacts GitHub, conservés 7 jours

Tableau 10 -- Étapes du pipeline CI/CD GitHub Actions

Emplacement visuel suggéré -- Figure 7

Schema du pipeline CI/CD : déclenchement sur push/pull request vers main, puis exécution parallèle ou séquentielle des étapes Build Android / Build iOS / Build Web, jusqu'à la publication des artefacts téléchargeables depuis l'onglet Actions de GitHub.

Un second pipeline, défini dans `codemagic.yaml`, complète ce dispositif en s'appuyant sur la plateforme Codemagic (instance `mac_mini_m1`), suivant une logique de build similaire (Android, iOS, Web). La coexistence de ces deux configurations offre une redondance utile en phase de développement et permet de comparer les temps de build entre les deux environnements d'intégration continue.

6.2 Hébergement et infrastructure

L'hébergement de la partie serveur repose entièrement sur l'infrastructure managée de Supabase (projet `ixrebfxxhfapndprujvt`), qui héberge la base de données PostgreSQL, le service d'authentification, le stockage de fichiers et l'API REST associée. Cette approche dispense le porteur de projet de la gestion directe de serveurs, de systèmes d'exploitation ou de correctifs de sécurité au niveau infrastructure, la responsabilité opérationnelle de ces couches étant assumée par le fournisseur cloud.

La distribution de l'application aux utilisateurs finaux diffère selon la plateforme : le build web généré par le pipeline CI/CD peut être publié sur un hébergeur statique (par exemple Firebase Hosting, GitHub Pages ou un hébergement web classique) ; les builds Android (APK) et iOS, en phase de mémoire de fin d'études, sont distribués sous forme d'artefacts téléchargeables ou installés manuellement, une publication sur le Google Play Store et l'App Store constituant une perspective naturelle d'évolution (cf. 7.4).

6.3 Sécurisation de l'application

La sécurisation de CleanCity repose sur plusieurs mécanismes complémentaires. Au niveau des données, les politiques Row Level Security détaillées au chapitre 3 constituent la première ligne de défense, indépendamment de toute logique applicative côté client. Au niveau des secrets et clés d'API, les identifiants sensibles -- jeton d'accès Mapbox (`MAPBOX_ACCESS_TOKEN`), clés Supabase, identifiants OneSignal -- sont injectés au moment de la compilation via le mécanisme `--dart-define` de Flutter plutôt que d'être codés en dur dans le dépôt source, limitant ainsi le risque de fuite via l'historique Git.

Une distinction importante est également maintenue entre la clé publique (anon key) utilisée par le client, dont les privilèges sont entièrement encadrés par les politiques RLS, et une éventuelle clé de service (service rôle), réservée à des opérations d'administration côté serveur et qui ne doit en aucun cas être embarquée dans le code client. Cette séparation garantit qu'une extraction du binaire de l'application ne permettrait pas, à elle seule, de contourner les règles d'accès définies au niveau de la base de données.

[Recommandation pour la soutenance : vérifier, avant la remise finale du mémoire, qu'aucune clé secrète ou mot de passe n'apparaît en clair dans le dépôt GitHub public, notamment dans l'historique des commits ; le cas échéant, effectuer une rotation des clés concernées.]

6.4 Perspectives de maintenance et évolution technique

La maintenabilité de la plateforme s'appuie sur les outils natifs de Supabase : consultation des journaux d'exécution (logs) pour le diagnostic d'incidents, et recours aux recommandations automatiques de sécurité et de performance (advisors) permettant d'identifier, par exemple, des politiques RLS manquantes ou des index sous-optimaux sur une table nouvellement créée. Ces outils constituent une base solide pour assurer, au-delà du cadre du mémoire, une exploitation en conditions réelles de la plateforme.

A moyen terme, plusieurs axes de maintenance évolutive se dessinent : mise en place d'une surveillance (monitoring) applicative dédiée, automatisation plus poussée des tests d'intégration au sein du pipeline CI/CD existant, et introduction progressive d'un mode de fonctionnement partiellement hors connexion pour les zones à faible connectivité, ces éléments étant développés plus en détail au chapitre 7.

Chapitre 7 -- Bilan, competences acquises et perspectives

7.1 Bilan technique du projet

Le projet CleanCity a permis de mettre en oeuvre, de bout en bout, une application multiplateforme complete reposant sur une architecture Backend-as-a-Service moderne. L'ensemble des spécifications fonctionnelles retenues au chapitre 2 a été implemente et validé par les scénarios de test présentés au chapitre 5 : authentication multi-canal, signalement géolocalisé avec photographie, prise en charge par un collecteur, reception et pesee par un centre de tri, système de points écologiques, demande de retrait Mobile Money, messagerie et espace d'administration.

Le choix architectural de s'appuyer sur Supabase et sur PostGIS s'est révélé pertinent au regard des contraintes de temps propres a un mémoire de licence, tout en permettant d'aborder des problematiques avancees de sécurité des données (Row Level Security) et de traitement géospatial, généralement réservées a des architectures plus complexes.

7.2 Limités identifiées

Plusieurs limités doivent etre reconnues avec objectivite. Le mécanisme de paiement Mobile Money repose, dans l'état actuel du développement, sur une validation manuelle par un administrateur plutot que sur une intégration directe avec une passerelle de paiement d'un opérateur (Orange Money, MTN Mobile Money), ce qui limité le passage a l'échelle du dispositif d'incitation économique. De meme, l'application ne proposé pas encore de mode de fonctionnement véritablement hors connexion, ce qui peut constituer un frein a l'adoption dans les zones les moins bien couvertes par les réseaux mobiles au Cameroun.

Sur le plan des tests, la couverture automatisee reste concentree sur la logique des modèles de données ; une extension vers des tests d'intégration automatisés (par exemple via des environnements Supabase locaux ephemerés) permettrait de sécuriser davantage les évolutions futures du schema de base de données et des politiques RLS.

7.3 Competences acquises

- Développement mobile et web multiplateforme avec Flutter et Dart, incluant la gestion d'état, la navigation declarative (go_router) et l'intégration de SDK tiers (Mapbox, OneSignal).
- Conception et administration d'une base de données relationnelle PostgreSQL, incluant la modélisation de données géospatiales avec PostGIS et l'écriture de fonctions SQL personnalisées (RPC).

- Mise en oeuvre concrete de mécanismes de sécurité au niveau des données (Row Level Security), au-dela des seuls contrôlés applicatifs.
- Pratique du versionnement collaboratif avec Git et GitHub, et mise en place d'un pipeline d'intégration et de déploiement continu (CI/CD) avec GitHub Actions.
- Élaboration d'une documentation technique structurée (diagrammes UML et C4, dictionnaire de données, cahier des charges) au service de la communication et de la maintenabilité du projet.
- Démarche de gestion de projet individuelle : planification, priorisation des fonctionnalités au regard d'un délai contraint, et rédaction d'un rapport académique structure.

7.4 Perspectives d'évolution

- Intégrer une véritable passerelle de paiement Mobile Money (API Orange Money / MTN MoMo) afin d'automatiser le versement des gains aux collecteurs sans validation manuelle systématique.
- Développer un mode de fonctionnement hors connexion (mise en cache locale des signalements et synchronisation différée), particulièrement adapté aux zones peri-urbaines et rurales à connectivité limitée.
- Enrichir l'espace d'administration d'un tableau de bord statistique (volumes collectes par zone et par période, délais moyens de traitement, taux de valorisation par type de déchet) à destination des autorités municipales partenaires.
- Étendre la couverture de tests automatisés, notamment par des tests d'intégration vérifiant systématiquement les politiques Row Level Security à chaque évolution du schéma.
- Envisager une publication officielle sur le Google Play Store et l'App Store, accompagnée d'une stratégie de sensibilisation citoyenne dans les villes camerounaises pilotes.

Conclusion générale

Ce mémoire a présenté la démarche complète de conception, de développement et de déploiement de CleanCity, une application répondant à une problématique concrète et documentée de la gestion des déchets urbains au Cameroun : l'absence d'outil numérique de suivi en temps réel entre citoyens, collecteurs, centres de tri et autorités. En s'appuyant sur une architecture Flutter et Supabase associant PostgreSQL, PostGIS et des politiques de sécurité Row Level Security, le projet démontre qu'une équipe de taille réduite, dans le cadre temporel d'une licence, peut mettre en oeuvre une plateforme fonctionnelle, sécurisée et déployable sur plusieurs cibles.

Au-delà de l'aspect technique, ce travail a constitué une occasion de mobiliser et de consolider un ensemble de compétences d'ingénierie logicielle -- de l'analyse des besoins à la mise en production -- tout en apportant une réponse concrète, même partielle, à un enjeu de salubrité publique dont

l'importance pour le développement urbain durable du Cameroun n'est plus à démontrer. Les perspectives d'évolution identifiées, notamment l'intégration réelle du paiement mobile et le fonctionnement hors connexion, tracent une feuille de route claire pour la poursuite éventuelle de ce projet au-delà du cadre académique.

Bibliographie

[Liste indicative a compléter et a mettre en forme selon la norme bibliographique exigée par l'établissement (APA, IEEE, etc.), avec les dates de consultation effectives.]

- Supabase Inc., Documentation officielle Supabase, <https://supabase.com/docs>
- Google LLC, Documentation officielle Flutter, <https://docs.flutter.dev>
- Dart Team, Documentation officielle du langage Dart, <https://dart.dev/guides>
- PostGIS Project Steering Committee, Documentation PostGIS, <https://postgis.net/documentation/>
- The PostgreSQL Global Development Group, Documentation PostgreSQL, <https://www.postgresql.org/docs/>
- Mapbox Inc., Documentation Mapbox (Static Tiles API), <https://docs.mapbox.com>
- GitHub Inc., Documentation GitHub Actions, <https://docs.github.com/actions>
- OWASP Foundation, OWASP Top Ten, <https://owasp.org/www-project-top-ten/>
- Banque mondiale, What a Waste 2.0 -- A Global Snapshot of Solid Waste Management to 2050, Washington D.C.
- ONU-Habitat, World Cities Report (édition consultée à préciser)
- Ministère de l'Environnement, de la Protection de la Nature et du Développement Durable (MINEPDED), Cameroun -- documents et rapports sectoriels (à référencer selon les sources effectivement consultées)

Annexes

Annexe A -- Extrait du schema de base de données (schema.sql)

L'intégralité du schema SQL (extensions, types énumérés, tables, index, fonctions, politiques RLS et configuration du stockage) est disponible dans le fichier lib/supabase/schema.sql du dépôt source du projet. Un extrait relatif a la table waste_requests est reproduit ci-dessous a titre illustratif :

```
create table public.waste_requests (  
  id uuid primary key default gen_random_uuid(),  
  generator_id uuid not null références public.users (id),  
  waste_type public.waste_type not null default 'mixed',  
  status public.waste_status not null default 'pending',  
  ...  
);
```

Annexe B -- Extrait du pipeline CI/CD (flutter_ci_cd.yml)

on:

```
push: { branches: [main] }  
pull_request: { branches: [main] }
```

jobs:

```
build_and_upload:  
  runs-on: macos-latest  
  steps: [checkout, setup-java-17, setup-flutter, pub-get,  
    build-apk, build-ios, build-web, upload-artifacts]
```

Annexe C -- Captures d'écran

Emplacement visuel suggere -- Figure A

Écran d'authentification / sélection du rôle (auth_screens.dart), a inserer depuis l'application.

Emplacement visuel suggere -- Figure B

Tableau de bord générateur avec liste des signalements et carte associée, a inserer depuis l'application.

Emplacement visuel suggere -- Figure C

Écran de reception et de pesee cote centre de tri (center_screens.dart), a inserer depuis l'application.

Emplacement visuel suggere -- Figure D

Espace d'administration (admin_screens.dart) : gestion des utilisateurs et des demandes de retrait, a inserer depuis l'application.